

Homework 3: Markov chains and hidden Markov models

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyang Slavchev
Görkem Kadir Solun

```
set.seed(42)
```

(6) Identify the most likely coding versus non-coding composition of a given DNA sequence

(6a) The Viterbi quantities for sequence TAAC

All probabilities mentioned are $\log_{10}$ -transformed.

The initial state probabilities (I), transition probabilities (T) and emission probabilities (E), that will be used to calculate the Viterbi quantities, $v_k(n)$, are shown below.

Initial state probabilities (I)	C-Region	NC-Region
	-0.4	-0.2

Transition probabilities (T)	C-Region	NC-Region
C-Region	-0.50	-0.15
NC-Region	-0.20	-0.40

Emission probabilities (E)	A	C	G	T
C-Region	-0.7	-0.5	-1.0	-0.4
NC-Region	-0.7	-0.2	-1.0	-1.0

For T at position 1, we should compute $P_{C-Region}(T, 1)$ and $P_{NC-Region}(T, 1)$ or in another notation $v_{C-Region}(1)$ and $v_{NC-Region}(1)$ for T observed.

$$P_{C-Region}(T, 1) = I_{C-Region} + E_{C-Region, T} = -0.4 - 0.4 = -0.8$$

$$P_{NC-Region}(T, 1) = I_{NC-Region} + E_{NC-Region, T} = -0.2 - 1.0 = -1.2$$

For A at position 2, we should compute $P_{C-Region}(A, 2)$ and $P_{NC-Region}(A, 2)$ or in another notation $v_{C-Region}(2)$ and $v_{NC-Region}(2)$ for A observed.

$$\begin{aligned} P_{C-Region}(A, 2) &= E_{C-Region, A} + \max \begin{cases} P_{C-Region}(T, 1) + T_{C-Region, C-Region} \\ P_{NC-Region}(T, 1) + T_{NC-Region, C-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -0.8 - 0.5 \\ -1.2 - 0.2 \end{cases} = -0.7 + \max \begin{cases} -1.3 \\ -1.4 \end{cases} = -2.0 \end{aligned}$$

$$\begin{aligned} P_{NC-Region}(A, 2) &= E_{NC-Region, A} + \max \begin{cases} P_{NC-Region}(T, 1) + T_{NC-Region, NC-Region} \\ P_{C-Region}(T, 1) + T_{C-Region, NC-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -1.2 - 0.4 \\ -0.8 - 0.15 \end{cases} = -0.7 + \max \begin{cases} -1.60 \\ -0.95 \end{cases} = -1.65 \end{aligned}$$

$P_{C-Region}(A, 2)$ comes from position 1 C-region. $P_{NC-Region}(A, 2)$ comes from position 1 C-region.

For A at position 3, we should compute $P_{C-Region}(A, 3)$ and $P_{NC-Region}(A, 3)$ or in another notation $v_{C-Region}(3)$ and $v_{NC-Region}(3)$ for A observed.

$$\begin{aligned} P_{C-Region}(A, 3) &= E_{C-Region, A} + \max \begin{cases} P_{C-Region}(A, 2) + T_{C-Region, C-Region} \\ P_{NC-Region}(A, 2) + T_{NC-Region, C-Region} \end{cases} \\ &= -0.7 + \max \begin{cases} -2.0 - 0.5 \\ -1.65 - 0.2 \end{cases} = -0.7 + \max \begin{cases} -2.50 \\ -1.85 \end{cases} = -2.55 \end{aligned}$$

$$P_{NC-Region}(A, 3) = E_{NC-Region, A} + \max \begin{cases} P_{NC-Region}(A, 2) + T_{NC-Region, NC-Region} \\ P_{C-Region}(A, 2) + T_{C-Region, NC-Region} \end{cases}$$

$$= -0.7 + \max \begin{Bmatrix} -1.65 - 0.4 \\ -2.0 - 0.15 \end{Bmatrix} = -0.7 + \max \begin{Bmatrix} -2.05 \\ -2.15 \end{Bmatrix} = -2.75$$

$P_{C-Region}(A, 3)$ comes from position 2 NC-region. $P_{NC-Region}(A, 3)$ comes from position 2 NC-region.

For C at position 4, we should compute $P_{C-Region}(C, 4)$ and $P_{NC-Region}(C, 4)$ or in another notation $v_{C-Region}(4)$ and $v_{NC-Region}(4)$ for C observed.

$$P_{C-Region}(C, 4) = E_{C-Region, C} + \max \begin{Bmatrix} P_{C-Region}(A, 3) + T_{C-Region, C-Region} \\ P_{NC-Region}(A, 3) + T_{NC-Region, C-Region} \end{Bmatrix}$$

$$= -0.5 + \max \begin{Bmatrix} -2.55 - 0.5 \\ -2.75 - 0.2 \end{Bmatrix} = -0.5 + \max \begin{Bmatrix} -3.05 \\ -2.95 \end{Bmatrix} = -3.45$$

$$P_{NC-Region}(C, 4) = E_{NC-Region, A} + \max \begin{Bmatrix} P_{NC-Region}(A, 3) + T_{NC-Region, NC-Region} \\ P_{C-Region}(A, 3) + T_{C-Region, NC-Region} \end{Bmatrix}$$

$$= -0.2 + \max \begin{Bmatrix} -2.75 - 0.4 \\ -2.55 - 0.15 \end{Bmatrix} = -0.2 + \max \begin{Bmatrix} -3.15 \\ -2.70 \end{Bmatrix} = -2.9$$

$P_{C-Region}(C, 4)$ comes from position 3 NC-region. $P_{NC-Region}(C, 4)$ comes from position 3 C-region.

The resulting Viterbi quantities, ($\log_{10}$ -transformed probabilities) are summarised in the table below.

	T	A	A	C
C-Region	-0.80	-2.00	-2.55	-3.45
NC-Region	-1.20	-1.65	-2.75	-2.90

The pointers (previous state visited) are summarised in the table below.

Pointers	T	A	A	C
C-Region	Not applicable	Position 1, C region	Position 2, NC region	Position 3, NC region

Pointers	T	A	A	C
NC-Region	Not applicable	Position 1, C region	Position 2, NC region	Position 3, C region

(6b) The most likely hidden path

The state in the last position, where C is observed, with the highest probability is NC-Region, with a probability of -2.9. Using the pointers summarised in the previous tables, the resulting shortest path is: C, NC, C, NC, or more precisely summarised in table format below:

Shortest path	Position 1	Position 2	Position 3	Position 4
Hidden variable state	C-Region	NC-Region	C-Region	NC-Region

(7) Predicting protein secondary structure using HMMs

(7a) Reading and storing data

Saving the tab-delimited files as dataframes

```
proteins_train <- read.delim("exercise/data/proteins_train.tsv", sep="\t", header = FALSE)
proteins_test <- read.delim("exercise/data/proteins_test.tsv", sep="\t", header = FALSE)
proteins_new <- read.delim("exercise/data/proteins_new.tsv", sep="\t", header = FALSE)
```

Renaming the columns.

```
colnames(proteins_train) <- c("Protein ID", "Amino acid sequence", "Secondary structure code")
colnames(proteins_test) <- c("Protein ID", "Amino acid sequence", "Secondary structure code")
colnames(proteins_new) <- c("Protein ID", "Amino acid sequence")
```

(7b) Initial state probabilities (I), transition probabilities (T) and emission probabilities (E)

```
L <- nrow(proteins_train)

aa_codes <- c("A", "C", "D", "E", "F", "G", "H", "I", "K", "L", "M", "N", "P", "Q", "R", "S", "T", "U", "V", "W")

ss_codes <- c("B", "C", "E", "G", "H", "I", "S", "T")
```

```

I_counts <- matrix(0, nrow = length(ss_codes), ncol = 1, dimnames = list(ss_codes))
T_counts <- matrix(0, nrow = length(ss_codes), ncol = length(ss_codes), dimnames = list(ss_codes, ss_codes))
E_counts <- matrix(0, nrow = length(ss_codes), ncol = length(aa_codes), dimnames = list(ss_codes, aa_codes))

for (protein in 1:L) {

  aa_seq <- proteins_train$`Amino acid sequence`[protein]
  ss_seq <- proteins_train$`Secondary structure code`[protein]

  # initialization counts, C is the secondary structure of the first amino acid
  first_ss <- substr(ss_seq, 1, 1)
  I_counts[first_ss,] <- I_counts[first_ss,] + 1

  l_seq = nchar(aa_seq)

  for (N in 1:l_seq) {
    aa_N <- substr(aa_seq, N, N)
    ss_N <- substr(ss_seq, N, N)

    # emission counts
    E_counts[ss_N, aa_N] <- E_counts[ss_N, aa_N] + 1

    # transmission counts
    if (N != 1) {
      T_counts[prev_ss_N, ss_N] <- T_counts[prev_ss_N, ss_N] + 1
    }
    prev_ss_N <- ss_N
  }
}

I <- I_counts/sum(I_counts)
E <- E_counts/rowSums(E_counts)
T <- T_counts/rowSums(T_counts)

```

(7c) The stationary distribution π

An ergodic Markov chain has a unique stationary distribution $\pi = (\pi_l)_{l \in [K]}$ such that

$$\lim_{n \rightarrow \infty} T_{kl}^n = \pi_l = \sum_{k \in [K]} \pi_k T_{kl}, \quad l \in [K], \quad \sum_{l \in [K]} \pi_l = 1$$

independent of the initial distribution I . In our situation:

$$\sum_{k=1}^8 \pi_k = 1$$

In matrix notation, π is the solution of $\pi^t = \pi^t T$. π is the eigenvector of the transpose of T corresponding to an eigenvalue of 1.

```
decomposed_T <- eigen(t(T))
idx_value1 <- which.min(abs(decomposed_T$values - 1))
vector_value1 <- decomposed_T$vectors[,idx_value1]

pi <- vector_value1/sum(vector_value1)

pi_matrix = matrix(pi, dimnames = list(ss_codes))
print((pi_matrix))
```

```
      [,1]
B 0.0116560594
C 0.2042297412
E 0.2094674769
G 0.0343029736
H 0.3395299222
I 0.0001018782
S 0.0889276425
T 0.1117843060
```

(7d) The latent state sequence Z using Viterbi algorithm

```
source("exercise/code/viterbi.r")
```

Warning: package 'dplyr' was built under R version 4.5.2

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
log_I <- log(I)
log_T <- log(T)
log_E <- log(E)

# for proteins_test
proteins_test_pred <- proteins_test
colnames(proteins_test_pred) <- c("ProteinID", "AminoAcids", "SecondaryStructure")

proteins_test_result <- viterbi(
  E = log_E,
  Tr = log_T,
  I = log_I,
  p = proteins_test_pred
)

# proteins_test_result as tsv file
write.table(proteins_test_result, file="proteins_test_pred.tsv",
            quote=FALSE, sep="\t", col.names = FALSE, row.names = FALSE)

# for proteins_new
proteins_new_pred <- proteins_new
colnames(proteins_new_pred) <- c("ProteinID", "AminoAcids")

proteins_new_result <- viterbi(
  E = log_E,
  Tr = log_T,
  I = log_I,
  p = proteins_new_pred
)

# protein_new_result as tsv file
write.table(proteins_new_result, file="proteins_new_pred.tsv",
            quote=FALSE, sep="\t", col.names = FALSE, row.names = FALSE)
```

(7e) Confidence intervals via bootstrapping

```
B <- 1000
n_proteins <- nrow(proteins_train)

I_boot <- array(0, dim = c(length(ss_codes), 1, B), dimnames = list(ss_codes, "Init", NULL))
T_boot <- array(0, dim = c(length(ss_codes), length(ss_codes), B), dimnames = list(ss_codes,
E_boot <- array(0, dim = c(length(ss_codes), length(aa_codes), B), dimnames = list(ss_codes,

set.seed(42)

for (b in 1:B) {
  # Sample with replacement
  resampled_indices <- sample(1:n_proteins, n_proteins, replace = TRUE)
  boot_data <- proteins_train[resampled_indices, ]

  # Initialize counts for this bootstrap iteration
  I_counts <- matrix(0, nrow = length(ss_codes), ncol = 1, dimnames = list(ss_codes, "Init"))
  T_counts <- matrix(0, nrow = length(ss_codes), ncol = length(ss_codes), dimnames = list(ss_
  E_counts <- matrix(0, nrow = length(ss_codes), ncol = length(aa_codes), dimnames = list(ss_

  # Calculate counts (similar to 7b)
  for (protein in 1:n_proteins) {
    aa_seq <- boot_data$`Amino acid sequence`[protein]
    ss_seq <- boot_data$`Secondary structure code`[protein]

    first_ss <- substr(ss_seq, 1, 1)
    I_counts[first_ss, ] <- I_counts[first_ss, ] + 1

    l_seq <- nchar(aa_seq)
    if (l_seq > 0) {
      # Split strings into vectors of characters for faster access
      aa_split <- strsplit(aa_seq, "")[[1]]
      ss_split <- strsplit(ss_seq, "")[[1]]

      for (N in 1:l_seq) {
        # Emission counts
        E_counts[ss_split[N], aa_split[N]] <- E_counts[ss_split[N], aa_split[N]] + 1

        # Transition counts
        if (N > 1) {
          T_counts[ss_split[N-1], ss_split[N]] <- T_counts[ss_split[N-1], ss_split[N]] + 1

```

```

    }
  }
}

# Calculate probabilities and store them in the b-th slice of the array
I_boot[, , b] <- I_counts / sum(I_counts)

# Handle potential division by zero if a state never appears in a bootstrap sample
row_sums_E <- rowSums(E_counts)
E_boot[, , b] <- ifelse(row_sums_E == 0, 0, E_counts / row_sums_E)

row_sums_T <- rowSums(T_counts)
T_boot[, , b] <- ifelse(row_sums_T == 0, 0, T_counts / row_sums_T)
}

# Compute the empirical 95% confidence intervals (2.5% and 97.5% quantiles)
# We use the apply() function over the 1st and 2nd dimensions (rows and columns)
I_lower <- apply(I_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
I_upper <- apply(I_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

T_lower <- apply(T_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
T_upper <- apply(T_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

E_lower <- apply(E_boot, c(1, 2), quantile, probs = 0.025, na.rm = TRUE)
E_upper <- apply(E_boot, c(1, 2), quantile, probs = 0.975, na.rm = TRUE)

# To verify
print(I_lower)

```

	Init
B	0
C	1
E	0
G	0
H	0
I	0
S	0
T	0

```
print(I_upper)
```

```
Init
B    0
C    1
E    0
G    0
H    0
I    0
S    0
T    0
```

(7f) Accuracy of predictions

```
n_test_proteins <- nrow(proteins_test_result)
accuracies <- numeric(n_test_proteins)

for (i in 1:n_test_proteins) {

  real_seq <- proteins_test_result[i, 3]
  pred_seq <- proteins_test_result[i, 4]

  real_chars <- strsplit(real_seq, "")[[1]]
  pred_chars <- strsplit(pred_seq, "")[[1]]

  L_seq <- length(real_chars)

  if (L_seq > 0) {
    matches <- sum(real_chars == pred_chars)
    accuracies[i] <- matches / L_seq
  } else {
    accuracies[i] <- NA # Handle edge case if a sequence is completely empty
  }
}

summary(accuracies)
```

	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
	0.007752	0.226253	0.322917	0.319801	0.407240	0.857143